cs144: Introduction to Computer Systems
Autumn 2024

Homework 5
(Assigned Nov 22)
Due Dec 06 11:59pm

Submit your work by committing the hw5 directory of CNETID-cs144-aut-24 svn repository. Note that hw5 directory contains link.txt and hw5.txt, where link.txt is for you to check the homework questions in canvas, and hw5.txt is for you to write down answers (or you can use PDF file format (e.g., hw5.pdf) for your answers, PDFs of scanned handwritten pages must not exceed 6 megabytes) No other file formats or filenames are acceptable, and no files besides hw5.txt or hw5.pdf will be graded. Not following directions will result in losing points.

Related lecture: mem1, mem2

(1) (20 points)

This question concerns internal and external fragmentation in a simple computer with 10MB of memory, using Dynamic Relocation (Base and Bound). Memory is used only with the granularity of megabytes, so for this question we can consider the physical addresses to be 0, 1, . . ., 9. Processes P1, P2, and P3 require only 1MB, 2MB, and 3MB, respectively. However, for a new to-be-run process, a contiguous span of 3MB will be allocated in the lowest possible addresses. 1MB of memory at address 5 is already in use (address ranges 0–4 and 6–9 are free). There is no process migration. Consider the question parts below as a sequence (i.e. part **B** assumes that the OS has dealt with the allocation requested for part **A**). Concisely explain your answers; a simple diagram may help. Write "n/a" if there is no meaningful answer.

A. The user wants to run P1. Is there memory for it, and if so, in what address range will P1 be loaded?

B. The user next wants to run P2. Can P2 be loaded, and if so, in what address range?

C. The user next wants to run P3. Can P3 be loaded, and if so, in what address range?

D. After the OS tries to run P1, P2, and P3, what is the internal fragmentation for each of the processes (P1, P2, and P3), and what is the total internal fragmentation?

E. After the OS tries to run P1, P2, and P3, what is the external fragmentation?

(2) (20 points)

Let's travel back to the past where the computer only provides **8-bit addressing and 2 segments**. The segments are code and stack only. The *code* segment's number is "0" and the *stack*'s segment number is "1". Please assume the stack offset arithmetic uses addition, not subtraction. (just like we do it in the class).

Let's suppose a process P1 is running and the segmentation table of P1 is as follows. (Do not worry about bounds in this question).

Segment	Base	R	W
Stack (1)	0x36	1	1
Code (0)	0xA8	1	0

A. Please convert the following memory accesses. We have provided you a table that helps you convert logical addresses into physical addresses (";" is just a column divider). Access can be read (r) or write (w). In the last column, please put "S" for success and "E" for error.

For the "offset bits" column, please put the actual bits (not hexadecimal numbers). For example, if you believe the offset width is 6 bits and the content is 0x17, then you need to write six bits "010111".

```
Access &
logical   Segment   Offset          Base       Offset     Physical    Success
address    bit(s)   bit(s)       (in hex)   (in hex)   address       or
                                                        (in hex)     Error

r 0xE5   =   ___ | _________   ;  _________ + _________ = _________   (____)

w 0x4A   =   ___ | _________   ;  _________ + _________ = _________   (____)

r 0x37   =   ___ | _________   ;  _________ + _________ = _________   (____)
```

B. Given the **architecture** above, what is the largest possible size of a segment? You can answer in any of the following forms: 2^k bytes or X bytes/KB/MB/GB (where X is a decimal number).

REMINDER - STARTING IN 2022, WE HAVE STRICTER POLICIES ON PROJECT AND HOMEWORK SUBMISSIONS:

Please triple check that you have submitted the right file(s), the right filename, the right content, the right format, and the right version.

We recommend submitting 15 minutes before the deadline (or even earlier) so you have 15 minutes to double/triple check your submission.

We will not accommodate incorrect or late submissions. 11:59:59pm is the hard deadline.